

CPU OS Simulator for Process Management and Scheduling

Introduction

Tool Based Learning is an innovative approach adopted in the Operating System course to enhance practical understanding of core OS concepts. Instead of learning only theoretical definitions, simulation tools are used to visualize how an operating system manages processes, schedules CPU time, handles synchronization, avoids deadlocks, and manages memory. This method helps students understand complex concepts in a simple and interactive manner.

Process Creation and Management:

Process management is one of the primary responsibilities of an operating system. A process is a program in execution. The operating system creates processes, allocates CPU time, manages their execution, and terminates them after completion.

Using a simulation tool, multiple processes can be created and observed as they move through different states such as **New, Ready, Running, Waiting, and Terminated**. The simulator clearly shows how the CPU switches between processes and manages execution efficiently.

This visualization helps in understanding multitasking and how the operating system controls multiple processes simultaneously.

CPU Scheduling Algorithms:

CPU scheduling is the process of selecting one process from the ready queue to allocate the CPU. Different scheduling algorithms are used depending on system requirements. Tool-based simulators help in comparing these algorithms using Gantt charts, waiting time, and turnaround time.

Comparison of CPU Scheduling Algorithms

Algorithm	Basic of selection	Advantage	Disadvantage
FCFS	Arrival Time	Simple to implement	High waiting time (Convoy Effect)
SJF	Shortest Burst Time	Minimum average waiting time	Starvation possible for long processes
Priority	Priority Value	Important tasks executed first	Low priority starvation
Round Robin	Time Quantum	Fair CPU sharing; Good for interactive systems	High context switching overhead

Process Synchronization:

Process synchronization is required when multiple processes access shared resources. Without synchronization, race conditions may occur, leading to incorrect results.

Using simulation tools and diagrams, the concept of **critical section**, **mutual exclusion**, and **race condition** can be easily understood. Synchronization mechanisms ensure that only one process accesses the shared resource at a time, maintaining data consistency.

Deadlock:

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource held by another process. Deadlock occurs due to conditions such as mutual exclusion, hold and wait, no preemption, and circular wait.

Through simulation and diagrams, deadlock situations can be visualized clearly. This helps in understanding why deadlocks occur and why deadlock prevention and avoidance techniques are necessary in an operating system.

Virtual Memory Management

Virtual memory allows execution of processes that may not be completely present in physical memory. Paging is a commonly used technique in virtual memory management.

Simulation tools demonstrate page allocation, page faults, and page replacement techniques. When a page fault occurs, the operating system replaces pages using suitable page replacement algorithms. This visualization helps in understanding memory optimization techniques used by modern operating systems.

Page Replacement Concepts

Term	Description
Page	Fixed-size block of memory
Frame	Physical memory block
Page fault	Required page not in memory
Page Replacement	Replacing old page with new page

Conclusion

Tool Based Learning helped in understanding Operating System concepts in a simple and practical way. By using simulation tools, topics such as process management, CPU scheduling, synchronization, deadlock, and virtual memory became clear and easy to visualize. This method reduced the difficulty of learning complex concepts only through theory.

Overall, tool based learning improved conceptual clarity, increased interest in the subject, and provided better insight into real operating system behavior. It effectively bridges the gap between theory and practical understanding.

Process Management and Multithreading

Introduction

An Operating System (OS) is a system software that acts as an interface between the user and the computer hardware. One of the most important responsibilities of an operating system is **process management** and **resource utilization**. Modern operating systems are designed to support multitasking, multiprocessing, and multithreading to improve system efficiency and performance.

Process management involves the creation, execution, scheduling, synchronization, and termination of processes. Multithreading allows a process to be divided into multiple independent threads that execute concurrently, sharing the same address space.

This tool based learning experiment focuses on understanding:

- Process creation using system calls
- Parent–child relationship between processes
- Synchronization using wait()
- Zombie and Orphan processes
- Matrix multiplication using multithreading

Aim:

To study and implement **process management** and **multithreading** concepts in an operating system using C programming by:

- Creating parent and child processes using fork()
- Sorting integers in parent and child processes
- Demonstrating Zombie and Orphan states
- Implementing matrix multiplication using POSIX threads

Objectives:

Sr. No.	Objective Description
1	To understand the concept of process creation
2	To implement the fork() system call
3	To study synchronization using wait()
4	To understand Zombie and Orphan processes
5	To study multithreading concepts
6	To implement matrix multiplication using threads
7	To analyze performance improvement using threads

Software and Hardware Requirements:

Component	Specification
Operating System	Linux / Ubuntu
Programming Language	C
Compiler	GCC
Thread Library	POSIX pthread
Tools	Terminal / OS Simulator
Hardware	Minimum 4GB RAM
Processor	Any modern processor

Theory:

1. Process

A **process** is an active instance of a program in execution. It contains:

- Program code
- Data
- Stack
- Heap
- Process Control Block (PCB)

Each process has a unique **Process ID (PID)** assigned by the operating system.

2. Process Management

Process management is a fundamental function of an operating system responsible for:

- Process creation
- Process scheduling
- Process synchronization
- Inter-process communication
- Process termination

The operating system maintains all process-related information using a **Process Control Block (PCB)**.

3. fork() System Call

The fork() system call is used to create a new process.

After calling fork():

- Parent and child execute the same code
- Return value differs:

0 → Child process

Child PID → Parent process

-1 → Failure

4. wait() System Call

The wait() system call is used by the parent process to wait until the child process finishes execution. It prevents zombie processes and ensures synchronization.

5. Zombie Process

A **Zombie process** is a process that has completed execution but still exists in the process table because its parent has not yet read its exit status using wait().

Characteristics:

- No CPU usage
- Occupies process table entry
- Removed only when parent calls wait()

6. Orphan Process

An **Orphan process** is a process whose parent terminates before the child. The child process is then adopted by the **init/systemd** process.

7. Multithreading

A **thread** is the smallest unit of execution within a process. Multithreading improves performance by allowing parallel execution of tasks.

Advantages of Multithreading:

- Faster execution
- Better CPU utilization
- Reduced response time
- Efficient resource sharing

Experiment 1: Process Management Using fork()

1. Problem Statement

Write a C program to read integers to be sorted. The main program spawns a child process using the fork() system call.

The parent process sorts the integers using Bubble Sort and waits for the child process using wait().

The child process sorts the integers using Selection Sort.

Also demonstrate Zombie and Orphan process states.

2. Algorithm

Step	Description
1	Read number of integers
2	Read array elements
3	Create child process using fork()
4	Parent sorts array using Bubble Sort
5	Child sorts array using Selection Sort
6	Child sleeps to demonstrate Zombie
7	Parent waits for child
8	Terminate processes

3. C Program: Process Management

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
void bubbleSort(int a[], int n) {
```

```
    int i, j, temp;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - i - 1; j++) {
```



```

        if (a[j] > a[j + 1]) {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}

```

```

void selectionSort(int a[], int n) {
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}

```

```

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

```

```
printf("Enter elements:\n");  
for (i = 0; i < n; i++)  
    scanf("%d", &arr[i]);  
  
pid_t pid = fork();  
  
if (pid == 0) {  
    printf("\nChild Process\n");  
    selectionSort(arr, n);  
    printf("Sorted by Child (Selection Sort): ");  
    for (i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
  
    sleep(5); // Zombie demonstration  
    exit(0);  
} else {  
    printf("\nParent Process\n");  
    bubbleSort(arr, n);  
    printf("Sorted by Parent (Bubble Sort): ");  
    for (i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
  
    wait(NULL);  
    printf("\nChild process terminated\n");  
    exit(0);  
}  
}
```

4. Explanation

- Parent and child processes execute concurrently
- Both sort the same input independently
- `sleep()` delays child termination
- `wait()` prevents zombie processes
- Demonstrates parent-child synchronization

Experiment 2: Matrix Multiplication Using Threads

1. Problem Statement

Write a C program to perform matrix multiplication using multithreading where each thread computes one row of the result matrix.

2. Algorithm

Step	Description
1	Read matrix dimensions
2	Validate multiplication condition
3	Read matrix elements
4	Create threads
5	Perform multiplication
6	Join threads
7	Display result

3. C Program: Multithreading

```
#include <stdio.h>

#include <pthread.h>

int A[10][10], B[10][10], C[10][10];
int r1, c1, r2, c2;

void* multiply(void* arg) {
    int row = (int)arg;
    int i, j;

    for (j = 0; j < c2; j++) {
        C[row][j] = 0;
        for (i = 0; i < c1; i++) {
            C[row][j] += A[row][i] * B[i][j];
        }
    }
    pthread_exit(NULL);
}

int main() {
    int i, j;
    pthread_t threads[10];

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d%d", &r1, &c1);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d%d", &r2, &c2);
```

```

if (c1 != r2) {
    printf("Multiplication not possible\n");
    return 0;
}

printf("Enter Matrix A:\n");
for (i = 0; i < r1; i++)
    for (j = 0; j < c1; j++)
        scanf("%d", &A[i][j]);

printf("Enter Matrix B:\n");
for (i = 0; i < r2; i++)
    for (j = 0; j < c2; j++)
        scanf("%d", &B[i][j]);

for (i = 0; i < r1; i++)
    pthread_create(&threads[i], NULL, multiply, (void*)i);

for (i = 0; i < r1; i++)
    pthread_join(threads[i], NULL);

printf("Result Matrix:\n");
for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++)
        printf("%d ", C[i][j]);
    printf("\n");
}

return 0;
}

```

Advantages

- Better CPU utilization
- Parallel execution
- Improved system performance
- Efficient resource sharing
- Reduced execution time

Applications

- Operating systems
- Scientific computing
- Web servers
- Database systems
- Real-time systems

Conclusion

Thus, the concepts of **process management and multithreading** were successfully studied and implemented. The use of `fork()`, `wait()`, and POSIX threads provided a clear understanding of process creation, synchronization, zombie and orphan states, and parallel execution. This experiment helps in understanding how modern operating systems efficiently manage resources.

CPU/Process Scheduling and Process Synchronization

Introduction

In modern computing systems, multiple processes often need to execute concurrently. The **Operating System (OS)** is responsible for managing these processes efficiently, ensuring fair allocation of CPU time, and maximizing system performance. **CPU/process scheduling** is a key function of the OS, which decides **the order in which processes get access to the CPU**. Efficient scheduling improves system throughput, reduces waiting and turnaround time, and ensures optimal utilization of CPU resources.

A process is an active program in execution. Each process may be in one of several states: **New, Ready, Running, Waiting, or Terminated**. The OS maintains a **Ready Queue** where processes wait for CPU allocation. The scheduling strategy employed by the OS determines **which process from the ready queue gets the CPU next**. Various scheduling algorithms, such as **First-Come-First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR)**, are used depending on system workload and process requirements. Each algorithm has its advantages and limitations, and the choice of algorithm affects metrics like **CPU utilization, turnaround time, waiting time, and throughput**.

Process synchronization is another critical aspect of operating systems. When multiple processes or threads share resources like memory, files, or I/O devices, proper synchronization mechanisms are necessary to **avoid conflicts, race conditions, and data inconsistency**. Threads are lightweight units of execution within a process that share the same memory space. **Multithreading** allows concurrent execution of tasks, improving CPU utilization and reducing overall execution time. **Synchronization tools**, such as **mutex locks, semaphores, and monitors**, ensure that multiple threads or processes access shared resources safely without interference.

Aim

To study and implement **CPU/process scheduling** and **process synchronization** using C programming by:

- Implementing CPU scheduling algorithms (FCFS, SJF, Priority, Round Robin)
- Comparing performance metrics: CPU utilization, waiting time, turnaround time, and throughput
- Implementing process synchronization using threads

Objectives:

Sr. No.	Objective
1	Understand CPU scheduling concepts and algorithms
2	Implement FCFS, SJF, Priority, and Round Robin scheduling
3	Analyze CPU utilization, turnaround time, waiting time, and throughput
4	Learn multithreading and process synchronization
5	Implement mutual exclusion using threads
6	Compare performance of different scheduling algorithms

Software and Hardware Requirements:

Component	Requirement
Operating System	Linux / Ubuntu
Programming Language	C
Compiler	GCC
Thread Library	POSIX pthread
Tools	Terminal / OS Simulator
RAM	4GB Minimum
Processor	Any modern CPU

Theory

1. CPU Scheduling

CPU scheduling is the process of deciding which ready process in memory will get CPU next. Efficient CPU scheduling improves system **throughput, CPU utilization, and response time**, and reduces waiting time.

Processes may be **preemptive** (interrupted before completion) or **non-preemptive** (run to completion). Scheduling algorithms determine the order of execution.

2. Scheduling Algorithms

1. First Come First Serve (FCFS)

- Non-preemptive
- Executes processes in order of arrival

2. Shortest Job First (SJF)

- Executes the process with minimum burst time
- Can be preemptive or non-preemptive

3. Priority Scheduling

- Processes assigned priorities
- Higher priority executes first
- Can be preemptive or non-preemptive

4. Round Robin (RR)

- Preemptive
- Each process executes for a fixed **time quantum**
- Processes are rotated in circular order

3. Performance Metrics

Metric	Description
CPU Utilization	Percentage of time CPU is busy
Turnaround Time (TAT)	Completion time – Arrival time
Waiting Time (WT)	Turnaround time – Burst time
Throughput	Number of processes completed per unit time

4. Process Synchronization

Process synchronization ensures **concurrent processes or threads** access shared resources safely, avoiding:

- **Race conditions** – simultaneous access causing incorrect results
- **Deadlocks** – two processes waiting for each other indefinitely
- **Starvation** – a process waiting forever for CPU

Synchronization mechanisms include **mutex locks, semaphores, and monitors**.

5. Threads

A thread is the **smallest unit of execution** within a process. Threads share memory and resources but have **independent execution contexts** (stack, program counter). Multithreading improves **parallel execution and CPU utilization**.

Experiment 1: CPU Scheduling Algorithms

1. Problem Statement

Implement FCFS, SJF, Priority, and Round Robin scheduling algorithms in C. Compare performance metrics: CPU utilization, TAT, WT, and throughput.

2. Algorithm (Stepwise)

Step	Description
1	Read number of processes
2	Read arrival and burst times
3	Implement FCFS and calculate TAT, WT
4	Implement SJF and calculate TAT, WT
5	Implement Priority scheduling and calculate TAT, WT
6	Implement Round Robin with time quantum
7	Calculate CPU utilization and throughput
8	Compare all algorithms in a table

3. C Program – FCFS Scheduling

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int bt[n], at[n], ct[n], tat[n], wt[n];

    for(i=0;i<n;i++){
        printf("Enter arrival and burst time of P%d: ",i+1);
        scanf("%d %d",&at[i],&bt[i]);
    }

    int time=0;
```

```

for(i=0;i<n;i++){
    if(time<at[i]) time=at[i];
    time+=bt[i];
    ct[i]=time;
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
}

printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0;i<n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);

return 0;
}

```

4. C Program – Round Robin Scheduling

```

#include <stdio.h>

int main() {
    int n, i, time=0, tq;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int bt[n], rem[n], at[n], wt[n]={0}, tat[n];
    for(i=0;i<n;i++){
        printf("Enter arrival and burst time of P%d: ",i+1);
        scanf("%d %d",&at[i],&bt[i]);
    }
}

```

```

        rem[i]=bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d",&tq);

    int done=0;
    while(done<n){
        done=0;
        for(i=0;i<n;i++){
            if(rem[i]>0){
                if(rem[i]>tq){ time+=tq; rem[i]-=tq; }
                else { time+=rem[i]; rem[i]=0; tat[i]=time-at[i]; wt[i]=tat[i]-bt[i]; }
            }
            if(rem[i]==0) done++;
        }
    }

    printf("\nP\tAT\tBT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],tat[i],wt[i]);
    return 0;
}

```

5. Explanation

- Each algorithm computes **TAT, WT, CPU utilization, throughput**
- RR uses **time quantum** for preemptive execution
- Comparison tables help analyze **performance efficiency**

Experiment 2: Process Synchronization Using Threads

1. Problem Statement

Implement mutual exclusion using **threads**. Multiple threads increment a **shared counter** safely using **mutex locks**.

2. Algorithm (Stepwise)

Step	Description
1	Initialize shared resource
2	Create multiple threads
3	Each thread increments shared variable
4	Apply mutex lock for critical section
5	Join threads and display final value

3. C Program – Thread Synchronization

```
#include <stdio.h>
#include <pthread.h>
int counter=0;
pthread_mutex_t lock;

void* increment(void* arg){
    for(int i=0;i<10000;i++){
        pthread_mutex_lock(&lock);
```

```

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main(){
    pthread_t t1,t2;
    pthread_mutex_init(&lock,NULL);

    pthread_create(&t1,NULL,increment,NULL);
    pthread_create(&t2,NULL,increment,NULL);

    pthread_join(t1,NULL);
    pthread_join(t2,NULL);

    printf("Final counter value: %d\n",counter);
    pthread_mutex_destroy(&lock);
    return 0;
}

```

4. Explanation

- Threads increment the same counter concurrently
- Mutex ensures **mutual exclusion**
- Prevents **race conditions** and guarantees correct final result

Advantages

- Better CPU utilization
- Reduced waiting & turnaround time

- Prevents race conditions

Applications

- Multitasking OS
- Real-time systems
- Web servers & database servers
- Scientific simulations
- Resource sharing in concurrent applications

Conclusion

This tool-based practical successfully demonstrated:

- Implementation of CPU scheduling algorithms
- Analysis of CPU utilization, TAT, WT, and throughput
- Process synchronization using threads and mutex locks
- Practical understanding of concurrency, multitasking, and synchronization

This experiment strengthens both **theoretical knowledge and practical programming skills** in operating systems.

References

1. Silberschatz, "Operating System Concepts," 10th Edition, Wiley
2. Tanenbaum, "Modern Operating Systems," 4th Edition, Pearson
3. Linux man pages (man pthread_create)

Deadlocks and Memory Management

Introduction

In modern operating systems, **deadlocks and memory management** are crucial for maintaining system stability and efficiency. Deadlocks occur when multiple processes compete for shared resources in such a way that none can proceed. Proper handling and prevention of deadlocks are essential to ensure smooth execution of concurrent processes.

Deadlocks arise when four conditions hold simultaneously:

1. **Mutual Exclusion:** A resource cannot be shared and is only available to one process at a time.
2. **Hold and Wait:** Processes holding resources can request additional resources.
3. **No Preemption:** Resources cannot be forcibly taken from processes.
4. **Circular Wait:** A circular chain of processes exists, each waiting for a resource held by the next.

Memory management is another critical OS function. Modern systems often use **virtual memory** to allow processes to run even if their data does not entirely fit in physical memory. Efficient memory management ensures fast access, prevents fragmentation, and maintains process isolation.

Paging is a common memory management technique that divides memory into fixed-size blocks called **pages** (logical) and **frames** (physical). Variations include **hierarchical paging, hashed page tables, and inverted page tables**. These methods improve **memory access efficiency and address translation**, especially for large memory systems.

In this tool-based practical, we implement:

- **Banker's Algorithm** to prevent deadlocks and allocate resources safely
- **Memory management simulator** to demonstrate paging techniques

These experiments provide **hands-on understanding** of resource allocation, safe execution, and memory handling in operating systems.

Aim

To study and implement **deadlock prevention and memory management techniques** using C programming by:

1. Implementing the **Banker's Algorithm** for safe resource allocation
2. Implementing a **paging simulator** to demonstrate **Hierarchical Paging / Hashed Page Tables / Inverted Page Tables**

Objective

Sr. No .	Objective
1	Understand deadlocks and their conditions
2	Implement Banker's Algorithm for safe resource allocation
3	Prevent unsafe states and deadlocks
4	Understand virtual memory and paging mechanisms
5	Implement a simulator for Hierarchical/Hashed/Inverted Page Tables
6	Analyze memory access and address translation in paging

Software and Hardware Requirements

Component	Requirement
Operating System	Linux / Ubuntu
Programming Language	C
Compiler	GCC
Tools	Terminal / OS Simulator
RAM	4GB Minimum
Processor	Any modern CPU

Theory

1. Deadlocks

A **deadlock** is a situation in which a set of processes are **blocked indefinitely**, each waiting for resources held by another. Deadlocks can severely degrade system performance.

Deadlock Handling Techniques:

1. **Deadlock Prevention:** Ensure at least one of the four necessary conditions cannot hold.
2. **Deadlock Avoidance:** Use resource allocation strategies like **Banker's Algorithm** to avoid unsafe states.
3. **Deadlock Detection & Recovery:** Allow deadlock and then detect & terminate or roll back processes.

2. Banker's Algorithm

- Proposed by **Edgar Dijkstra**
- Ensures that resource allocation is **safe**
- Checks if granting a request leaves system in a **safe state**
- **Safe state:** There exists a sequence of process execution such that all processes can finish

Algorithm Steps:

1. Maintain **Available, Max, Allocation, Need** matrices
2. Check if requested resources $\leq$ Available
3. Temporarily allocate resources and check for safe sequence
4. If safe, allocate; else, process waits

3. Memory Management & Paging

Memory management is required to efficiently store and retrieve process data. **Paging** divides memory into fixed-size blocks:

- **Pages:** Logical memory units
- **Frames:** Physical memory units

Types of Page Tables:

1. **Hierarchical Paging:** Multi-level page tables to reduce memory for large address spaces
2. **Hashed Page Tables:** Hash function maps virtual pages to table entries
3. **Inverted Page Tables:** One table entry per physical frame, reducing memory size for large processes

Paging allows **non-contiguous memory allocation**, prevents fragmentation, and simplifies memory access.

Experiment 1: Banker's Algorithm

1. Problem Statement

Implement **Banker's Algorithm** in C to allocate resources safely and prevent deadlocks.

2. Algorithm (Stepwise)

Step	Description
1	Read number of processes and resources
2	Read Available , Max , and Allocation matrices
3	Calculate Need matrix = Max – Allocation
4	Check if requested resources can be safely allocated
5	If safe, allocate resources; else process waits
6	Display safe sequence of processes

3. C Program – Banker's Algorithm

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j;
```

```

printf("Enter number of processes and resources: ");
scanf("%d %d",&n,&m);

int alloc[n][m], max[n][m], avail[m], need[n][m], finish[n], safe[n];

printf("Enter Allocation Matrix:\n");
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        scanf("%d",&alloc[i][j]);

printf("Enter Max Matrix:\n");
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        scanf("%d",&max[i][j]);

printf("Enter Available Resources: ");
for(i=0;i<m;i++) scanf("%d",&avail[i]);

// Calculate Need matrix
for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        need[i][j]=max[i][j]-alloc[i][j];
for(i=0;i<n;i++) finish[i]=0;
int count=0;
while(count<n){
    int found=0;
    for(i=0;i<n;i++){
        if(finish[i]==0){
            int can_allocate=1;

```

```

    for(j=0;j<m;j++)
        if(need[i][j]>avail[j]) { can_allocate=0; break; }
    if(can_allocate){
        for(j=0;j<m;j++) avail[j]+=alloc[i][j];
        safe[count++]=i;
        finish[i]=1;
        found=1;
    }
}
}
}
if(found==0){
    printf("System is in unsafe state!\n");
    return 0;
}
}

printf("System is in safe state.\nSafe sequence: ");
for(i=0;i<n;i++) printf("P%d ",safe[i]);
return 0;
}

```

4. Explanation

- Computes **Need matrix** = Max – Allocation
- Iteratively checks which process can complete safely
- Allocates resources only if system remains **safe**
- Displays **safe sequence**, preventing deadlocks

Experiment 2: Paging Simulator

1. Problem Statement

Implement a simulator in C for **Hierarchical Paging** (or Hashed/Inverted Page Table).
Demonstrate **virtual to physical address translation**.

2. Algorithm (Stepwise)

Step	Description
1	Define page size and number of pages/frames
2	Create page table (single/multi-level)
3	Input virtual addresses to access
4	Translate virtual addresses to physical addresses using page table
5	Handle page faults and update table

3. C Program – Hierarchical Paging Simulator (Simplified)

```
#include <stdio.h>

#define PAGES 16
#define FRAMES 8
#define PAGE_SIZE 1024

int main() {
    int page_table[PAGES];
    int i;

    printf("Enter frame number for each page (-1 if not allocated):\n");
    for(i=0;i<PAGES;i++) scanf("%d",&page_table[i]);

    int va, page, offset;

    printf("Enter virtual address to translate: ");
    scanf("%d",&va);
```

```
page = va / PAGE_SIZE;
offset = va % PAGE_SIZE;

if(page_table[page]==-1)
    printf("Page fault!\n");
else
    printf("Physical Address: %d\n",page_table[page]*PAGE_SIZE + offset);

return 0;
}
```

4. Explanation

- User enters **frame numbers** for each page
- Virtual address is divided into **page number** and **offset**
- Physical address is calculated as (frame_number * page_size + offset)
- Handles **page faults** if page not allocated

Advantages

- Prevents deadlocks using safe resource allocation
- Efficient memory management with paging
- Reduces fragmentation and ensures process isolation
- Demonstrates real-world OS concepts practically

Applications

- Multitasking operating systems

- Database servers for concurrent access
- Memory management in large address space systems
- Safe resource allocation in real-time systems
- Virtual memory systems in modern CPUs

Conclusion

This tool-based practical successfully demonstrated:

- **Deadlock avoidance using Banker's Algorithm**
- **Safe sequence calculation and resource allocation**
- **Memory management and paging simulation**
- **Virtual to physical address translation**

Students gained **hands-on experience in deadlocks, resource allocation, and memory management**, reinforcing core OS concepts in theory and programming.

References

1. Silberschatz, "Operating System Concepts," 10th Edition, Wiley
2. Tanenbaum, "Modern Operating Systems," 4th Edition, Pearson
3. Linux man pages (man malloc, man pthread)

File System Management

Topic: File Allocation Methods

Introduction

File System Management is an important part of an Operating System. It is responsible for storing, organizing, managing, and retrieving files on secondary storage devices such as hard disks and SSDs.

One of the major tasks of file system management is deciding **how files are stored on disk**. This is done using **file allocation methods**.

File allocation methods determine how disk blocks are assigned to files and how these blocks are accessed efficiently.

What is File Allocation?

File allocation is the process of assigning disk space to files.

When a file is created, the operating system allocates disk blocks to store the file's data.

When the file is deleted, the allocated disk blocks are freed.

Objectives of File Allocation

- Efficient use of disk space
- Fast access to files
- Minimum disk fragmentation
- Easy file expansion

Types of File Allocation Methods

There are **three main file allocation methods** used by operating systems:

1. Contiguous Allocation
2. Linked Allocation
3. Indexed Allocation

Contiguous Allocation

1. Description

In contiguous allocation, all blocks of a file are stored **in consecutive (continuous) disk blocks**.

Example:

If a file needs 5 blocks and starts at block 20, then it occupies blocks 20–24.

Disk Blocks →

| 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |

File A → Blocks 20 to 24

2. Advantages

- Simple to implement
- Fast access (direct access supported)
- Minimal seek time

3. Disadvantages

- External fragmentation occurs
- Difficult to increase file size
- Requires knowing file size in advance

Use Case : Used in systems where file sizes are fixed, such as **CD-ROMs**.

Linked Allocation

1. Description

In linked allocation, file blocks are stored **anywhere on disk**, and each block contains a pointer to the next block.

Block 5 → Block 19 → Block 8 → Block 26 → NULL

2. Advantages

- No external fragmentation
- Files can grow easily
- Efficient disk space utilization

3. Disadvantages

- Slow access (no direct access)
- Extra space needed for pointers
- Pointer corruption can cause data loss

Use Case : Used in **FAT (File Allocation Table)** file systems.

Indexed Allocation

1. Description

In indexed allocation, each file has an **index block** that stores the addresses of all disk blocks allocated to that file.

Index Block →

| 12 | 25 | 30 | 41 |

Actual Data Blocks →

12 25 30 41

2. Advantages

- Supports direct and sequential access
- No external fragmentation
- Easy file expansion

3. Disadvantages

- Extra overhead for index block
- Small files waste index block space

Use Case : Used in **UNIX/Linux file systems** (inode structure).

Comparison Table

Feature	Contiguous	Linked	Indexed
Fragmentation	External	No	No
Direct Access	Yes	No	Yes
File Growth	Difficult	Easy	Easy
Access Speed	Fast	Slow	Fast
Space Utilization	Medium	High	High

C Program: Indexed File Allocation

```
#include <stdio.h>

int main() {
    int files, i, j;
    int indexBlock[10][10];
    int blocks[10];

    printf("Enter number of files: ");
    scanf("%d", &files);

    for (i = 0; i < files; i++) {
```

```

        printf("\nEnter number of blocks for file %d: ", i + 1);
        scanf("%d", &blocks[i]);

        printf("Enter block numbers:\n");
        for (j = 0; j < blocks[i]; j++) {
            scanf("%d", &indexBlock[i][j]);
        }
    }
    printf("\nFile Allocation Details:\n");
    for (i = 0; i < files; i++) {
        printf("File %d blocks: ", i + 1);
        for (j = 0; j < blocks[i]; j++) {
            printf("%d ", indexBlock[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Sample Output :

```

Enter number of files: 2
Enter number of blocks for file 1: 3
Enter block numbers: 5 10 15
Enter number of blocks for file 2: 2
Enter block numbers: 7 20

```

File Allocation Details:

File 1 blocks: 5 10 15

File 2 blocks: 7 20

Conclusion

File allocation methods are essential for efficient file system management.

Each method has advantages and limitations.

Among all, **indexed allocation** is widely used in modern operating systems due to its flexibility, efficient access, and reduced fragmentation.